CAMPUS DIGITAL SERVICE

Campus Lost & Found

Plan complet de conception UX/UI responsive
Mobile - Tablette - Desktop

Projet	Application et infrastructure locale auto-hebergee
Equipe	Groupe de 3 membres
Stack cible	Angular - NestJS - PostgreSQL - Docker - Nginx

Version 1.0 - 21 septembre 2026

01 - CADRAGE

Vision produit et objectifs

Une plateforme universitaire simple, rassurante et rapide pour signaler, retrouver et restituer un objet.



Objectif utilisateur

Comprendre l'action principale en moins de 5 secondes et terminer une publication ou une reclamation sans aide.

Objectif metier

Tracer chaque etape: annonce, reclamation, verification, rendez-vous, code et restitution.

Objectif projet Cloud

Livrer une application complete, testee, conteneurisee et reproductible sur une VM Linux.

Principes UX

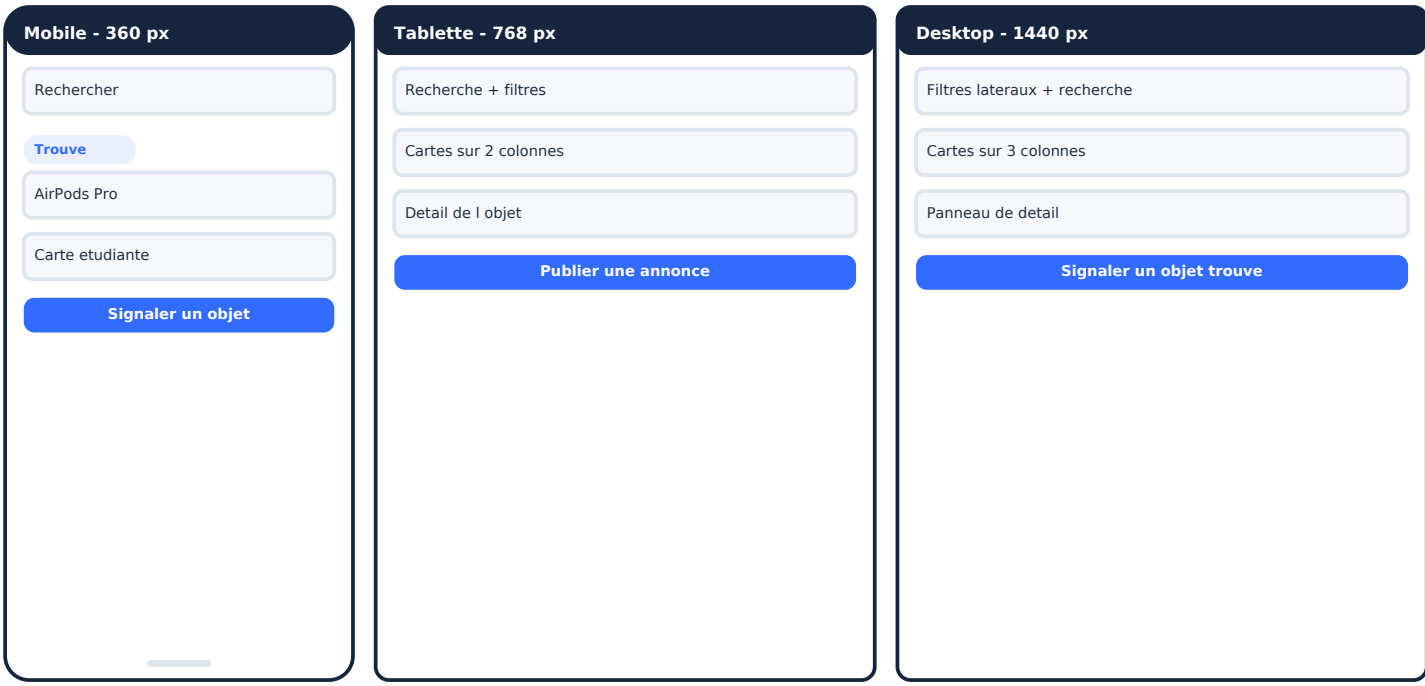
Principe	Application concrete
Une action dominante	Deux CTA visibles: Signaler une perte / Signaler un objet trouve.
Divulgation progressive	Les preuves secretes ne sont jamais affichees dans l'annonce publique.
Confiance	Statuts explicites, confirmation des deux parties et code de restitution.
Accessibilite	Contrastes lisibles, zones tactiles >= 44 px, libelles visibles et navigation clavier.
Responsive	Meme parcours sur mobile, tablette et desktop; seule la disposition change.

Perimetre MVP: authentification, deux roles, CRUD de plusieurs entites liees, recherche/filtres/pagination, upload d'image et tableau de bord administrateur.

02 - RESPONSIVE

Une interface adaptee a chaque ecran

Approche mobile-first: concevoir d’abord pour 360 px, puis enrichir progressivement.



Zone	Mobile < 600 px	Tablette 600-1023 px	Desktop >= 1024 px
Navigation	Barre basse: Accueil, Recherche, Publier, Activite, Profil	Barre haute + navigation compacte	Header complet + actions + panneau de filtres
Grille	1 carte par ligne	2 cartes par ligne	3 a 4 cartes par ligne
Filtres	Panneau coulissant plein ecran	Panneau lateral temporaire	Sidebar fixe
Formulaires	1 colonne, CTA fixe en bas	1-2 colonnes	2 colonnes + resume lateral
Detail objet	Contenu empile	Image + informations	Image, informations et action en panneau lateral

Regle: aucune fonctionnalite ne doit exister uniquement sur desktop. Sur mobile, les menus secondaires peuvent etre replies, mais restent accessibles.

03 - IDENTITE VISUELLE

Design system simple et coherent

Style recommande: moderne, universitaire, rassurant, avec des cartes claires et beaucoup d’espace blanc.

Couleur	Code	Usage
Navy	#15253E	Navigation, titres
Blue	#316BFF	CTA principal, liens
Teal	#18A999	Succes, objet rendu
Gold	#F4B942	Attention, attente
Red	#E35D6A	Erreur, refus
Cloud	#F6F8FC	Arriere-plan

Typographie

Titres: Inter / DejaVu Sans Bold
Texte: Inter / DejaVu Sans
Echelle: 32, 24, 20, 16, 14 px

Espacement

Unite: 4 px
Grille: 8, 16, 24, 32 px
Rayon: 12 px cartes, 8 px champs

Iconographie

Icones simples avec libelle. Ne jamais utiliser la couleur seule pour exprimer un statut.

Composants reutilisables

Composant	Etats obligatoires
Bouton	Default, hover, focus, loading, disabled
Champ	Vide, rempli, focus, erreur, aide
Carte objet	LOST/FOUND, actif, reclame, rendu
Badge statut	ACTIVE, CLAIMED, RETURNED, CLOSED
Toast / alerte	Succes, information, avertissement, erreur
Modal de confirmation	Suppression, refus, confirmation de remise
Skeleton / vide	Chargement, aucune donnee, erreur reseau

04 - ARCHITECTURE DE L'INFORMATION

Navigation claire par role

Utilisateur non connecte

Ecran	Contenu / action
Accueil	Recherche globale, annonces recentes, CTA perte/trouve
Liste des objets	Recherche, filtres, tri, pagination
Detail objet	Photos, lieu, date, categorie, statut; connexion requise pour reclamer
Authentification	Inscription, connexion, mot de passe oublie si retenu

Utilisateur connecte

Ecran	Contenu / action
Publier	Assistant en 3 etapes: informations, photo/preuves, verification
Mes annonces	Modifier, fermer, consulter les reclamations
Mes reclamations	Suivre PENDING, ACCEPTED, REJECTED, COMPLETED
Restitution	Rendez-vous, code de recuperation, double confirmation
Notifications	Nouvelle reclamation, decision, rendez-vous, restitution
Profil	Identite, avatar, preferences, deconnexion

Administrateur

Ecran	Contenu / action
Dashboard	Compteurs, tendances, repartition par categorie et statut
Moderation	Annonces, utilisateurs, reclamations et signalements
Referentiels	CRUD categories et lieux

05 - ECRANS PUBLICS

Accueil, recherche et fiche objet

Accueil

Header: logo, recherche, connexion.
Hero: “Perdu quelque chose?” + deux CTA.
Sections: trouves recemment, comment ca marche, conseils.

Liste

Recherche: titre/mot-cle.
Filtres: type, categorie, lieu, date, statut.
Tri: plus recent / plus pertinent.
Pagination: numerique.

Detail

Visible: photo, description publique, date, lieu approximatif, auteur, statut.
Action: Reclamer cet objet.
Prive: details de verification.

Carte d’objet - contenu minimal

Element	Regle UX
Photo	Ratio 4:3; placeholder si absente; texte alternatif.
Type	Badge bleu “PERDU” ou vert “TROUVE”.
Titre	Deux lignes maximum; pas de details secrets.
Meta	Categorie, lieu, date avec icones + texte.
Statut	Toujours visible: Actif, Reclame, Rendu, Ferme.
Action	“Voir les details”; toute la carte peut aussi etre cliquable.

Etats a dessiner dans Figma avant le code

- Chargement: cartes skeleton.
- Liste vide: message utile + bouton “Modifier les filtres”.
- Erreur API: message simple + bouton “Reessayer”.
- Connexion lente: conserver les valeurs des formulaires localement.

06 - PUBLICATION

Un formulaire en 3 etapes

1 Informations publiques → 2 Photo et details prives → 3 Verification et publication

Etape	Champs	Regles
1. Informations publiques	Type LOST/FOUND, titre, description, categorie, lieu, date	Champs obligatoires marques; date non future pour un objet trouve.
2. Photo et verification	Image, details secrets, question de verification	Image JPG/PNG, taille limite; details secrets chiffres cote serveur.
3. Resume	Apercu public, rappel des informations masquees, consentement	Bouton Publier; retour possible sans perte de donnees.



Validation immediate

Afficher l’erreur sous le champ apres interaction. Ne pas effacer les donnees si l’upload ou l’API echoue.

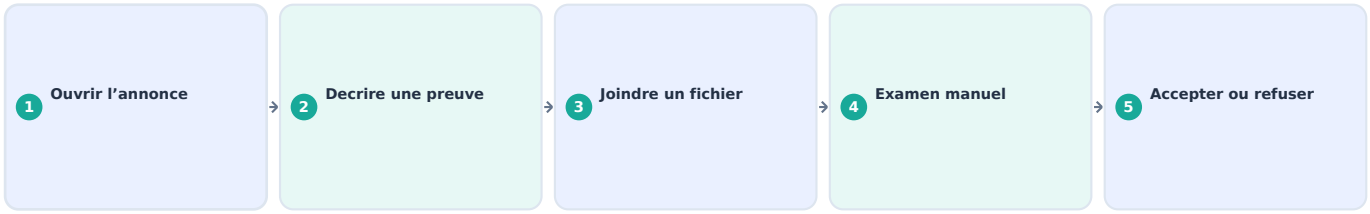
Confidentialite

Le champ verificationDetails ne doit jamais etre present dans la reponse de l’API publique. Les preuves jointes doivent etre accessibles uniquement aux personnes autorisees.

07 - RECLAMATION ET VERIFICATION

Verifier la propriete sans promettre une certitude automatique

La decision est manuelle: le finder ou un administrateur compare les preuves et les details secrets.



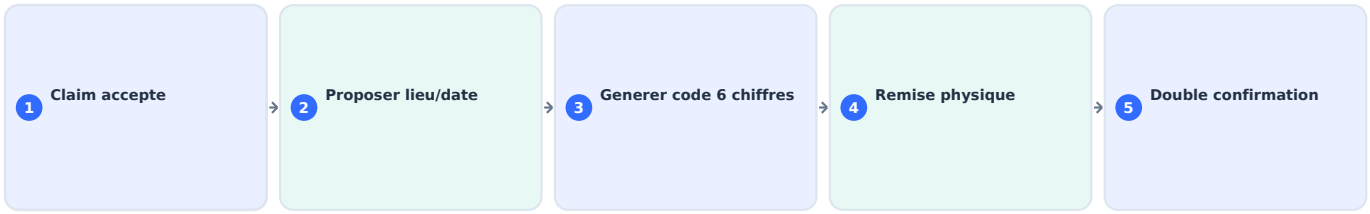
Ecran demandeur	Ecran finder / admin
Formulaire: preuve textuelle, reponse a une question, fichier optionnel.	Liste des reclamations avec identite, date, message et piece jointe.
Message de confidentialite et rappel de ne pas envoyer de mot de passe.	Comparaison avec les details prives saisis lors de la publication.
Suivi du statut et historique horodate.	Actions Accepter / Refuser avec confirmation et motif optionnel.
Si accepte: acces au rendez-vous et au code.	Une seule reclamation peut etre acceptee par objet.

Regles metier critiques

- Le demandeur ne peut pas reclamer sa propre annonce.
- Seuls l’auteur, le demandeur concerne et l’admin voient les preuves.
- Accepter une reclamation passe l’objet a CLAIMED et bloque les autres acceptations.
- Refuser ne ferme pas l’annonce; elle reste ACTIVE.
- Chaque decision est journalisee pour la tracabilite.

08 - RESTITUTION

Du rendez-vous au statut RETURNED



Rendez-vous securise

Choisir un lieu universitaire predetermine: accueil, administration, securite ou bibliotheque. Eviter les adresses personnelles.

Code de recuperation

Code aleatoire a 6 chiffres, expire apres usage. Il est affiche au proprietaire et saisi par le finder lors de la remise.

Confirmation finale

Le finder confirme “Objet remis” et le demandeur confirme “Objet recupere”. Le backend passe alors le claim a COMPLETED et l’objet a RETURNED.

Machine d’etats

Entite	Etat initial	Transition	Etat suivant
Claim	PENDING	Finder accepte	ACCEPTED
Item	ACTIVE	Un claim est accepte	CLAIMED
Claim	ACCEPTED	Code valide + confirmations	COMPLETED
Item	CLAIMED	Restitution terminee	RETURNED
Claim	PENDING	Finder refuse	REJECTED
Item	ACTIVE	Auteur ferme sans restitution	CLOSED

Important: le code confirme le rendez-vous et la remise, mais il ne prouve pas a lui seul la propriete. La verification repose sur les preuves examinees avant acceptation.

09 - ESPACES PRIVES

Dashboard utilisateur et administration

Dashboard utilisateur

Bloc	Contenu
Resume	Mes annonces actives, reclamations en attente, objets recuperes.
Actions rapides	Publier une perte, publier un objet trouve, lancer une recherche.
Activite recente	Dernieres notifications et changements de statut.
Mes annonces	Cartes avec modifier, fermer et voir les reclamations.

Dashboard administrateur

Zone	Indicateurs / actions
KPI	Utilisateurs, objets perdus, trouves, rendus, taux de restitution.
Graphiques	Objets par categorie; annonces par semaine; repartition des statuts.
Moderation	Derniers signalements, annonces suspectes, utilisateurs desactives.
Administration	CRUD categories/lieux; recherche utilisateurs; suppression annonce.

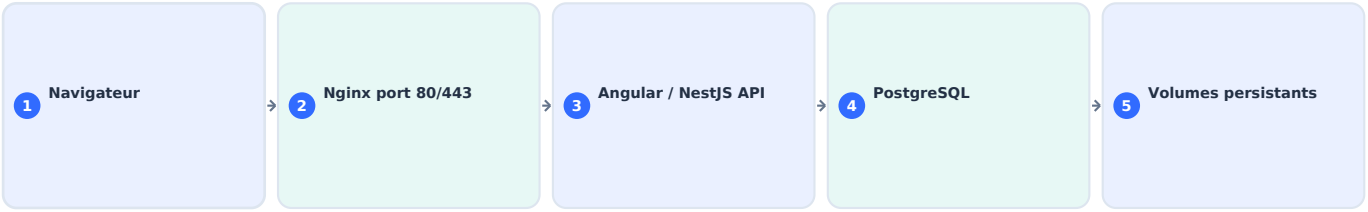
Calculs proposes

Indicateur	Formule
Taux de restitution	Nombre d’objets RETURNED / nombre total d’objets publies x 100
Temps moyen de restitution	Moyenne(completedAt - createdAt) pour les objets RETURNED
Taux d’acceptation	Claims ACCEPTED ou COMPLETED / claims examines x 100

10 - ARCHITECTURE TECHNIQUE

Application puis infrastructure locale

Couche	Choix	Responsabilite
Frontend	Angular	UI responsive, routes, formulaires, guards, services API.
Backend	NestJS	API REST, validation, permissions, logique metier, Swagger.
Donnees	PostgreSQL + Prisma/TypeORM	Relations, transactions, migrations et statistiques.
Fichiers	Multer + volume uploads	Images et preuves persistantes apres redemarrage.
Proxy	Nginx	Point d’entree unique; routage / vers frontend et /api vers backend.
Execution	Docker Compose sur VM Ubuntu	Frontend, backend, database et proxy sur reseau prive.
CI	GitHub Actions ou GitLab CI	Build + tests a chaque push; pas de CD obligatoire.



Reseau

Seul Nginx expose un port sur la VM. PostgreSQL reste accessible uniquement depuis le reseau Docker interne.

Persistance

Deux volumes nommes minimum: postgres_data et uploads_data. Tester un arret/redemarrage complet.

11 - DONNEES, API ET SECURITE

Contrats a definir avant les ecrans avances

Entite	Relations principales
User	1-N Item; 1-N Claim; 1-N Notification
Item	N-1 User; N-1 Category; N-1 Location; 1-N Claim
Claim	N-1 Item; N-1 claimant; rendez-vous; confirmations; code hashe
Category / Location	1-N Item; geres par ADMIN
Notification	N-1 User; lien vers la ressource concernee
Report (bonus)	N-1 Item; N-1 reporter; statut de moderation

API MVP

Module	Routes essentielles
Auth	POST /auth/register, POST /auth/login
Items	GET/POST /items, GET/PATCH/DELETE /items/:id
Claims	POST /items/:id/claims, GET /claims/my, PATCH accept/reject
Recovery	PATCH meeting, POST verify-code, PATCH confirm
Admin	GET statistics; CRUD categories/locations; moderation

Securite minimale

- bcrypt pour les mots de passe; JWT court avec secret dans .env.
- Validation DTO cote backend et controle MIME/taille des uploads.
- Guards de role et controle de propriete sur chaque modification.
- Rate limiting sur login, reclamations et verification du code.
- Ne jamais journaliser mots de passe, JWT, preuves privees ou code de restitution.

12 - ORGANISATION DU GROUPE

Repartition fonctionnelle pour 3 membres

Chaque membre livre frontend, backend et tests sur son perimetre; les taches Cloud sont reparties.

Membre	Perimetre principal	Livrables
Membre 1	Authentification et utilisateurs	Auth NestJS, JWT/roles, login/register/profile Angular, guards, Swagger, tests auth, Docker frontend.
Membre 2	Objets, recherche et uploads	Items/categories/lieux, recherche/filtres/pagination, formulaires/liste/detail, upload, tests items, Docker backend + DB.
Membre 3	Claims, restitution et admin	Claims/preuves, rendez-vous/code/confirmations, notifications, dashboard/admin, tests, Compose + VM + Nginx.
Tous	Architecture et qualite	MCD, conventions UI, revues de PR, integration, test E2E, README, demo.

Regles Git

Branche	Usage
main	Stable et protegee; versions validees.
develop	Integration des fonctionnalites terminees.
feature/auth, feature/items, feature/claims...	Une fonctionnalite courte par branche.
Pull request	Au moins un autre membre relit; CI verte avant merge.

Definition d’une tache terminee

- UI responsive testee a 360, 768 et 1440 px.
- Validation backend et autorisations implementees.
- Cas nominal + erreurs + etat vide couverts.
- Tests automatiques passes et pull request relue.
- Documentation mise a jour si contrat API ou exploitation change.

13 - PLANNING

Roadmap sur 12 semaines

Sem.	Objectif	Resultat attendu
S1	Validation sujet, personas, parcours, MCD, depot, choix VM	Architecture et backlog valides.
S2	Design system, maquettes mobile, auth et structure du projet	Prototype UI + inscription/connexion.
S3	Items, upload, recherche, filtres, pagination	Parcours publication/recherche fonctionnel.
S4	Claims, restitution minimale, tests, integration	Jalon 1: application locale demonstrable.
S5	Correction UX, admin, notifications, stabilisation	MVP fonctionnel et coherent.
S6	Dockerfiles frontend/backend/database	Images locales reproductibles.
S7	Compose, volumes, reseau, migrations	Stack demarree par une commande.
S8	VM, SSH, ufw, Nginx, test de persistance	Jalon 2: infrastructure demontree.
S9	Workflow Git, nettoyage .env, protection branches	Collaboration tracable.
S10	Pipeline build + tests	CI verte a chaque push.
S11	README, runbook, sauvegarde/restauration	Nouveau contributeur autonome.
S12	Tests finaux, repetition demo, soutenance	Jalon 3 et livraison finale.

Priorisation

MUST - MVP	SHOULD - apres MVP	COULD - bonus
Auth, roles, objets, recherche, upload, claims, restitution, dashboard, tests	Notifications temps reel, favoris, historique detaille, carte interne du campus	Matching IA/image, QR de restitution, PWA hors ligne

14 - VALIDATION

Checklist avant developpement et demonstration

Zone	Critere de validation
UX/UI	Maquettes mobile/tablette/desktop; design system; etats chargement/vide/erreur; navigation clavier.
Fonctionnel	2 roles; CRUD de 3+ entites; recherche/filtres/pagination; upload; dashboard.
Metier	Details secrets proteges; une seule claim acceptee; double confirmation; historique des statuts.
Backend	DTO valides; guards; controle de propriete; erreurs coherentes; Swagger.
Tests	Unitaires auth/items/claims; integration login, create item, claim, restitution.
Docker	Un Dockerfile par service; Compose; reseau prive; DB non exposee; utilisateur non-root recommande.
Persistance	Base et fichiers encore presents apres redemarrage complet de la stack et de la VM.
Exploitation	Nginx unique point d’entree; .env.example; CI build/tests; README; runbook; backup/restore.
Equipe	Commits significatifs pour chacun; PR relues; demo repetee en 10 minutes.

PROCHAINE ETAPE

Creer les wireframes dans Figma puis initialiser le monorepo avec frontend/, backend/, nginx/, docs/ et docker-compose.yml.

Source de cadrage: “Cahier des Charges - Projet Application et infrastructure locale auto-hebergee”, TEK-UP University, branche Cloud Computing. Le present plan adapte ces exigences au sujet Campus Lost & Found.